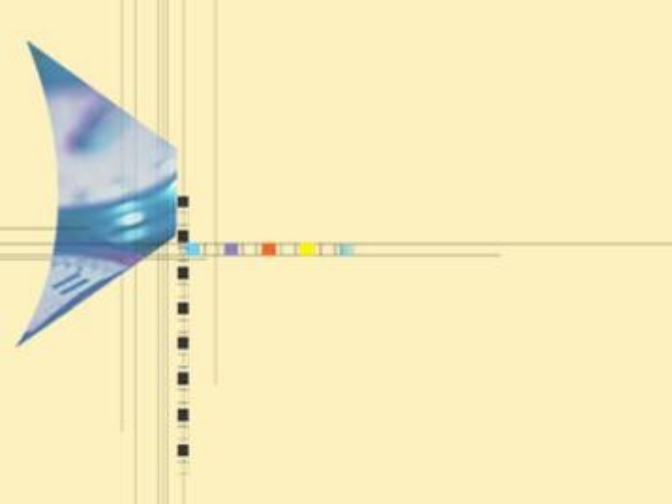




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

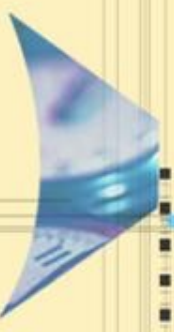
周一(3-4，三区1-502)，周三（6-7，三区1-325）

2025/4/18

武汉大学计算机学院

武汉大学
Wuhan University





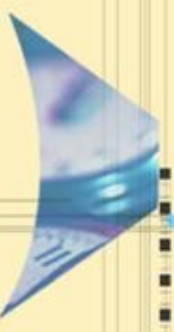
主要内容

8.1 引言

8.2 回溯法简单例

8.3 一般回溯法

8.4 回溯法的典型应用



主要内容

8.4 回溯法典型应用（选讲）

8.4.1 子集和问题

8.4.2 哈密顿回路问题

8.4.3 装载问题

8.4.4 0/1背包问题

8.4.5 流水作业调度问题

8.4.6 货郎担问题

8.4.1 子集和问题

- 已知 $n+1$ 个正数: $w_i, (1 \leq i \leq n)$, 和 M , 要求找出 w_i 的和数是 M 的所有子集
- 解的表示形式:
 - 采用定长元组: 解为 n 元组 $(x_1, x_2, \dots, x_n)$, 其中 x_i 取0或1, 表示 w_i 的在子集合中是否出现。
- 约束条件:

$$\sum_{i=1}^n w(i)x(i) = M$$

约束函数

(1) $B_k(x(1), \dots, x(k)) = \text{true}$ (表示 $(x_1, \dots, x_k)$ 为部分可行解) 当且仅当:

$$\sum_{i=1}^k w(i)x(i) + \sum_{i=k+1}^n w(i) \geq M$$

(2) 假定 w_i 已经按照非降次序排列, 则可以强化约束函数, 如果下式成立, $B_k(x(1), \dots, x(k))$ 为假 (表示 $(x_1, \dots, x_k)$ 不是部分可行解):

$$\sum_{i=1}^k w(i)x(i) + w(k+1) > M$$

综合 (1) (2), 故, 假定 w_i 已经按照非降次序排列, 约束函数 $B_k(x(1), \dots, x(k)) = \text{true}$ 当且仅当:

$$\sum_{i=1}^k w(i)x(i) + \sum_{i=k+1}^n w(i) \geq M \text{ and } \sum_{i=1}^k w(i)x(i) + w(k+1) \leq M$$

算法SUMOFSUBREC

输入: $w[1,2,\dots,n]$ and M , $w[i]>0$, $M > 0$.

输出: A solution vector $x[1,2,\dots,n]$, where each $x[i]$ is 0 or 1.

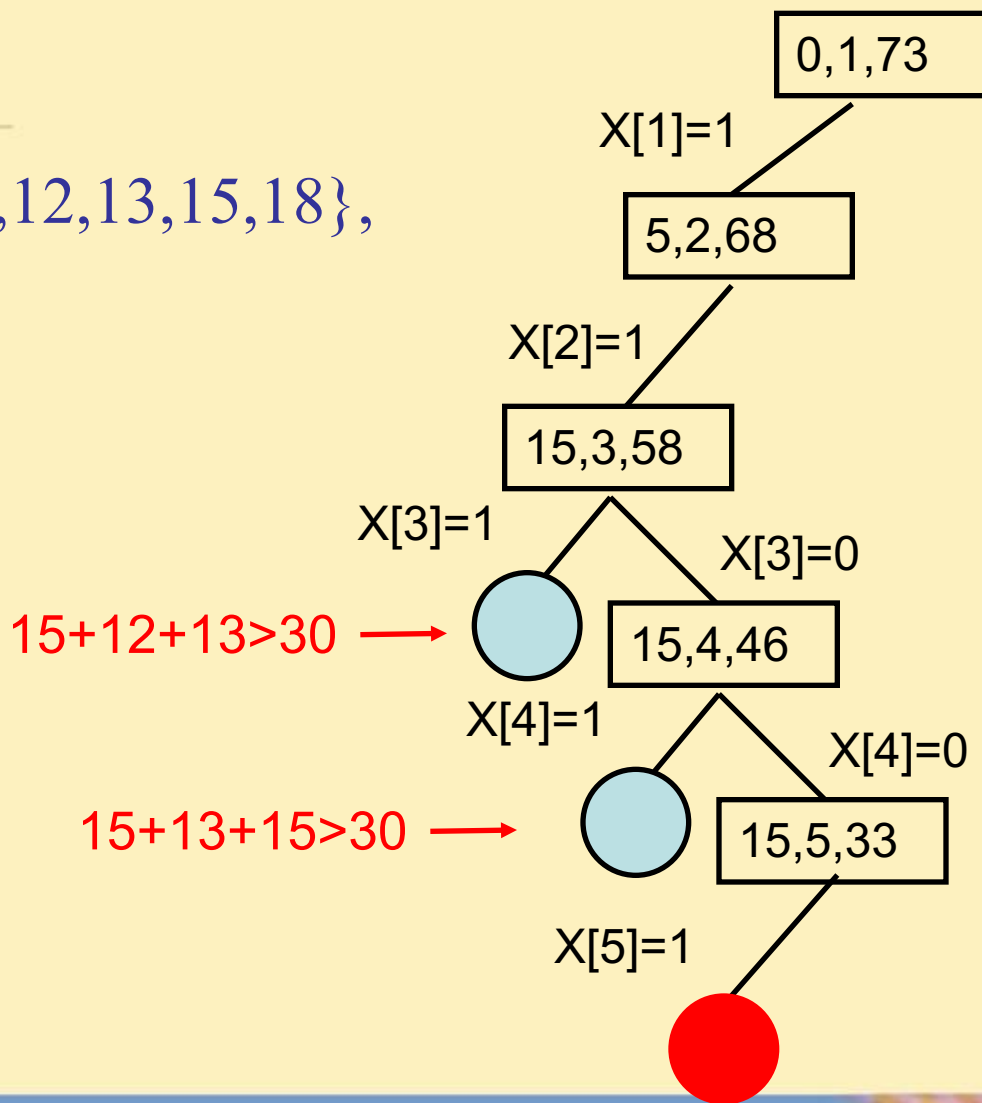
```
1. for  $k \leftarrow 1$  to  $n$ 
2.    $x[k] \leftarrow 0$ 
3. end for
4.  $flag \leftarrow false$ 
5.  $s \leftarrow 0$ ;  $r \leftarrow w[1]+w[2]+\dots+w[n]$ 
6. If  $r \geq M$  then SUMOFSUB( $s, 1, r$ ) endif
7. if  $flag$  then output  $x$ 
8. else output "no solution"
9. end if
```

Procedure SUMOFSUB (s, k, r)

```
1.  $x[k] \leftarrow 1$ 
2. if  $s+w[k]=M$  //x 是合法解
3. then set  $flag \leftarrow true$  and exit
4. else
5.   if  $s+w[k]+w[k+1] \leq M$  //x 是部分的  $B_k=true$ , 第k个数选入
6.   then SUMOFSUB( $s+w[k], k+1, r-w[k]$ ) //递归调用第k+1个数
7.   end if
8. end if
9. if  $s+r-w[k] \geq M$  and  $s+w[k+1] \leq M$  //  $B_k=false$  第k个数不选入
10. then  $x[k] \leftarrow 0$ , SUMOFSUB( $s, k+1, r-w[k]$ )
11. end if
```

实例

- $w[1..6]=\{5,10,12,13,15,18\}$,
 $M=30$



8.4.2 哈密顿回路问题

- 哈密顿回路: 设 $G=(V,E)$ 是一个 n 结点的连通图. 一个哈密顿回路是一条沿着图 G 的 n 条边环行的路径,它访问 V 中每个结点一次并且回到它的原始位置.
- 解的表示: 用定长的 n -元组 $(x_1, \dots, x_n)$ 表示解, 其中 x_i 是找到的回路中第 i 个被访问的结点。 $x_i \in \{1, 2, \dots, n\}$
- 约束条件: $x_1=1$, 如果 $1 < k < n$, 则 x_k 可以是不同于 $x_1, \dots, x_{k-1}$ 且和 x_{k-1} 有边相连的任一结点; x_n 是必须与 x_{n-1} 和 x_1 有边相连的结点。

$$x_1 = 1$$

$$(x_i, x_{i+1}) \in E, 1 \leq i < n$$

$$(x_n, x_1) \in E$$

$$x_i \neq x_j, 1 \leq i, j \leq n, i \neq j$$

算法 HAMILTONIANREC

Input: An undirected graph $G=(V,E)$.

Output: A solution vector $x[1,2,\dots,n]$, where $x[i]$ is the number of i th visited node, and $x[1]=1$.

1. **for** $k \leftarrow 1$ **to** n
2. $x[k] \leftarrow 0$
3. **end for**
4. $flag \leftarrow false$
5. HAMILTON(1)
6. **if** $flag$ **then output** x
7. **else output** “no solution”
8. **end if**

过程 HAMNILTONT(k)

1. **while true**
2. NEXTVALUE(k) //assign $x[k]$
3. **if** $x[k]=0$ **then exit** **end if**
4. **if** $k=n$ **then** $flag \leftarrow true$ **and exit**
5. **else** HAMILTON($k+1$)
6. **end if**
7. **end while**



过程 NEXTVALUE(k)

while true

$x[k] = x[k] + 1 \bmod (n+1)$

 if $x[k] = 0$ then exit end if

 if GRAPH ($x[k-1]$, $x[k]$)

 then

$j = 1$

 while $j < k$

 if $x[j] = x[k]$ then exit end if

$j++$

 end while

 if $j = k$

 then if $k < n$ or ($k = n$ and GRAPH($x[n]$, 1)) then exit end if

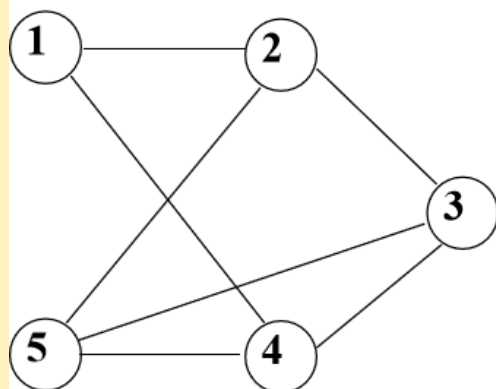
 end if

 end if

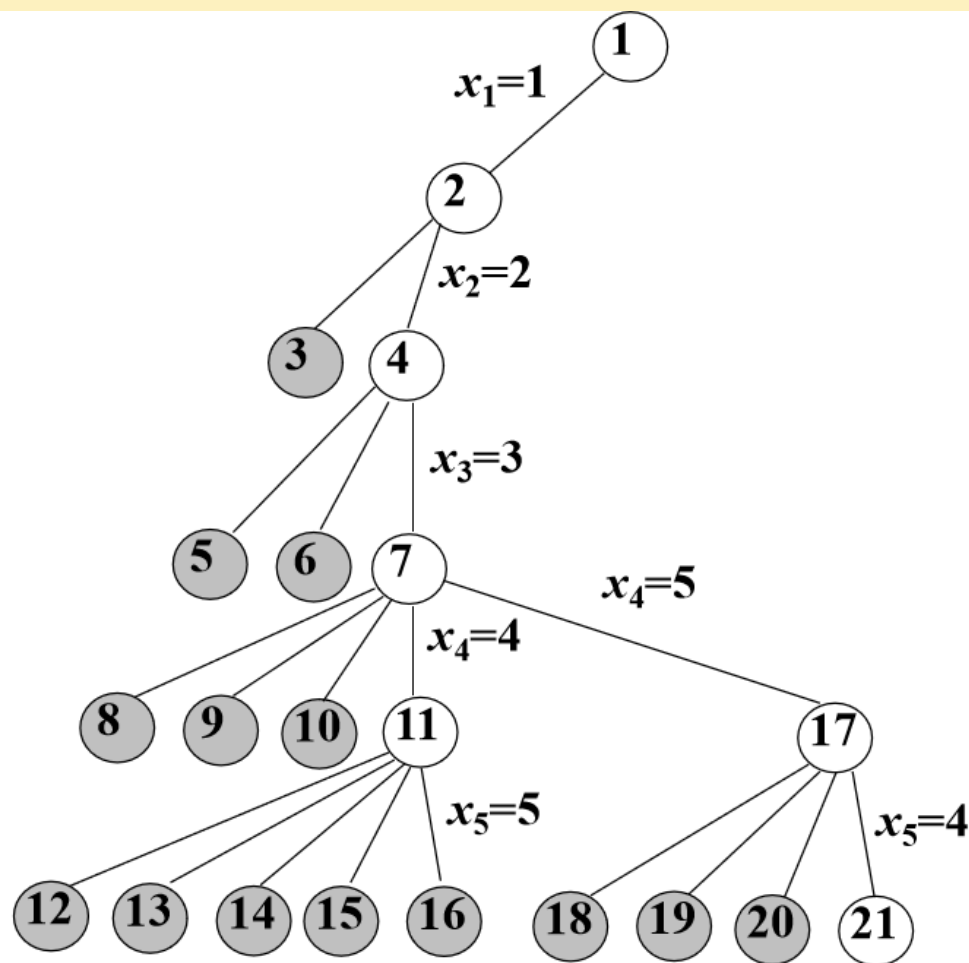
 end while



实例



(a) 一个无向图



(b) 哈密顿回路的搜索空间

https://blog.csdn.net/nice__amusin

8.4.3 装载问题

装载问题： n 个集装箱装上2艘载重分别为 c_1 和 c_2 的轮船， w_i 为集装箱 i 的重量， 且

$$\sum_{i=1}^n w_i \leq c_1 + c_2$$

问是否存在一种合理的装载方案将 n 个集装箱装上轮船？ 如果有， 给出一种方案.

可以证明， 如问题有解， 则一定存在一个使得第一条船装载量与 c_1 的差达到最小的解， 因此， 确定如下求解思路：

- (1) 令第一船装载量为 W_1 ， 用回溯算法求使 $c_1 - W_1$ 达到最小的装载方案 $\langle x_1, x_2, \dots, x_n \rangle$ ， （使第一条船达到最大装载量的方案）
- (2) 如果 $\sum_{i=1}^n w_i - W_1 \leq c_2$ 则回答Yes, 否则回答No.

8.4.3 装载问题

第一条船的最优装载方案求解

解向量形式： n -元组 $(x_1, \dots, x_n)$ ，其中 $x_i \in \{0, 1\}$ ，表示第一条船是否装载第 i 个集装箱。搜索空间是子集树。

约束条件：对结点 $(x_1, \dots, x_k)$ ，要求 $\sum_{i=1}^k w_i x_i \leq c_1$

容易看出，约束条件满足多米诺性质



算法 Loading (W, c_1),

输入：集装箱重量 $W = \langle w_1, w_2, \dots, w_n \rangle$, c_1 是第一条船的载重

输出：使第一条船装载最大的方案 $\langle x_1, x_2, \dots, x_n \rangle$, 其中 $x_i = 0, 1$

1. Sort(W); //对 $w_1, w_2, \dots, w_n$ 按照从大到小排序

2. $B \leftarrow c_1$; $best \leftarrow c_1$; $cw \leftarrow 0$; $i \leftarrow 1$;

3. while $i \leq n$ do

4. if $cw + w_i \leq c_1$ then

5. $B \leftarrow B - w_i$; $x[i] \leftarrow 1$; $cw \leftarrow cw + w_i$;

6. else $x[i] \leftarrow 0$;

7. end if

8. $i \leftarrow i + 1$;

9. end while

10. if $B < best$ then

11. 记录解; $best \leftarrow B$; $i \leftarrow i - 1$;

12. end if

11. Backtrack(i);

12. if $i = 1$ then return 最优解

13. else goto 4.

14. end if



过程 Backtrack(i)

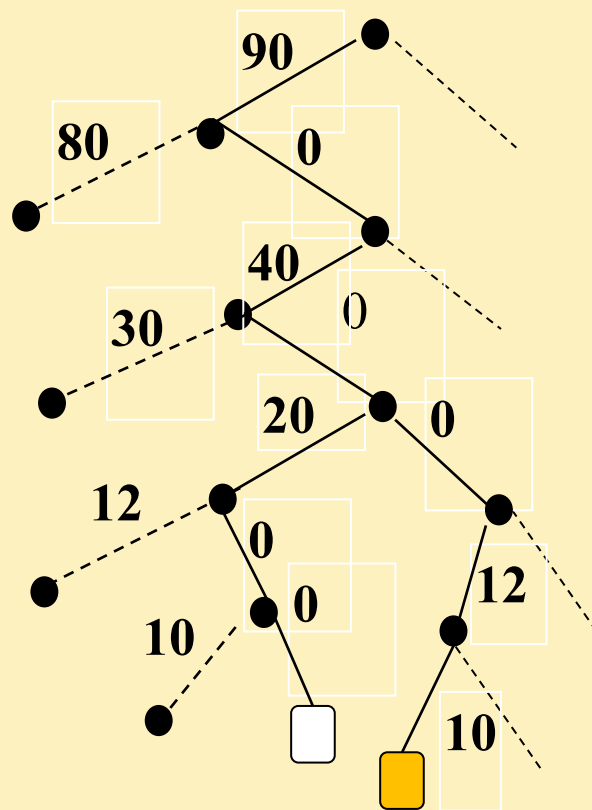
1. while $i > 1$ and $x[i] = 0$
2. $i \leftarrow i - 1$;
3. end while
4. if $x[i] = 1$ then
5. $x[i] \leftarrow 0$; $B \leftarrow B + w_i$; $i \leftarrow i + 1$;
6. end if

Loading 算法复杂性: $W(n) = O(2^n)$

$$c_1=152, c_2=130$$

第二个可行解(1,0,1,0,0,1,1)，装载量为152，最大装载

第一条船装载后，货物剩下的重量为 $80+30+20=130$ ，正好装入第二条船。



8.3.4 0/1背包问题

- 回顾0/1背包问题：有n个物品，大小分别是 $s_1, s_2, \dots, s_n$ ，价值分别为 $v_1, v_2, \dots, v_n$ ，背包总容量为C，均为正整数。选择其中若干物品放入背包，是的放进背包中的物品总重量不超过C，且总价值最大。
- 解的表示形式：采用定长的n元组 $(x_1, x_2, \dots, x_n)$ 表示，其中 x_i 取值0或者1，表示第i个物品是否装进背包中。
- 约束优化条件：

$$\sum_{1 \leq i \leq n} s_i x_i \leq C \quad \text{并且} \quad \max(\sum_{1 \leq i \leq n} v_i x_i)$$

强化约束-限界函数

- 对优化问题，可以通过定义界限函数，帮助杀死某些活节点，使其不再扩展。
- 0/1背包问题的限界函数：取成能产生某些值的上界函数，如果扩展给定的结点和它的任一子孙所导致的可行解值的上界不大于迄今所确定的最好解的值，就可以杀死此结点。
- 定义思路：假设当前确定的部分元组 $(x_1, x_2, \dots, x_k)$ ，则其所对应的结点Z的上界可由下法获得：对 $k+1 \leq i \leq n$ ，将 $x_i=0$ 或者1的要求放宽为 $0 \leq x_i \leq 1$ ，然后利用分数背包问题的贪心算法求解放宽要求的问题的解。则此解表示在k+1级上扩展任何一个结点所能得到的最好解的上界。

算法 BT-KNAPSACK

输入: n 个物品的集合 $U=\{u_1, u_2, \dots, u_n\}$, 对应的大小为 $W(1:n)$, 价值为 $P(1:n)$, 背包容量 M 。 n 个物品按 $P[i]/W[i]$.非升序排列

输出: 解向量 $X[1..n]$, fp, fw , 其中 $X[i]$ 为1或者0, 表示物品 u_i 在或者不在背包中, fw, fp : 最后装进背包中物品的总大小和总价值

注释: cw, cp : 当前背包中物品的大小和价值.

1. $fp \leftarrow -1, cp \leftarrow 0, cw \leftarrow 0$
2. $k \leftarrow 1$
3. **while true**
4. **while** $k \leq n$ and $cw + W(k) \leq M$ //将物品 u_k 放入背包
5. $cw \leftarrow cw + W[k]; cp \leftarrow cp + P[k]; Y[k] \leftarrow 1; k \leftarrow k + 1$ //advance
6. **end while**
7. **if** $k > n$ **then** $fp \leftarrow cp; fw \leftarrow cw; k \leftarrow n; X \leftarrow Y$ //修改解 X 为目前得到的最好解
8. **else** $Y[k] \leftarrow 0$ //如果放入 u_k , 总重量超出 M , 物品 u_k 不放入背包
9. **end if**
10. **while** $BOUND(cp, cw, k, M) \leq fp$ // $Y[k]=0$ 节点的上界。上面置了 fp 后, $BOUND=fp$
11. **while** $k \neq 0$ and $Y[k] \neq 1$
12. $k \leftarrow k - 1$ //回溯, 找最后放入背包物品
13. **end while**
14. **if** $k = 0$ **then** exit two while loops **end if** //算法结束
15. $Y[k] \leftarrow 0; cw \leftarrow cw - W[k]; cp \leftarrow cp - P[k]$ //从背包中移走物品 u_k
16. **end while**
17. $k \leftarrow k + 1$ //advance
18. **end while**
19. **output** X, fp, fw

限界函数

Procedure BOUND (p,w,k,M)

//p:当前总效益; w:当前总大小; k:上次去掉的物品;M:背包容量;返回新效益值

$b \leftarrow p; c \leftarrow w$

for $i \leftarrow k+1$ to n

$c \leftarrow c + W[i]$

if $c < M$ then $b \leftarrow b + P[i]$

else return $b + (M - (c - W[i])) * P[i] / W[i]$

end if

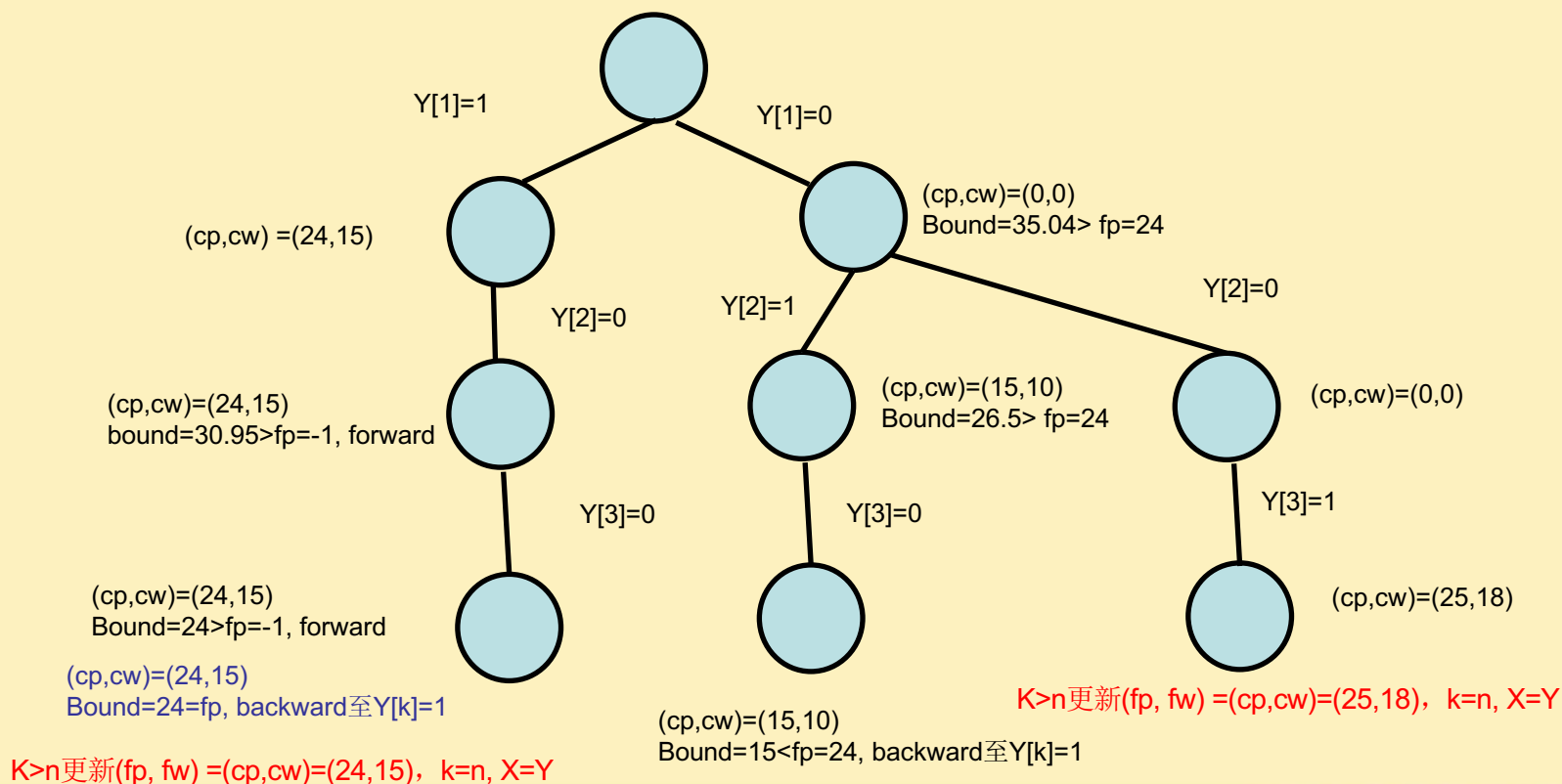
end for

return b

实例

- $n=3, C=20, P[1..3]=[24,15,25], W[1..3]=[15,10,18], P[i]/W[i]=(1.6, 1.5, 1.39)$

初始 $k=1, (fp, fw) = (-1, /); (cp, cw)=(0,0)$



8.3.5 流水作业调度问题

【问题描述】有 n 个作业（编号为 $1 \sim n$ ）要在由两台机器 M_1 和 M_2 组成的流水线上完成加工。每个作业加工的顺序都是先在 M_1 上加工，然后在 M_2 上加工。 M_1 和 M_2 加工作业 i 所需的时间分别为 a_i 和 b_i （ $1 \leq i \leq n$ ）。

流水作业调度问题要求确定这 n 个作业的最优加工顺序，使得从第一个作业在机器 M_1 上开始加工，到最后一个作业在机器 M_2 上加工完成所需的时间最少。可以假定任何作业一旦开始加工，就不允许被中断，直到该作业被完成，即非优先调度。

问题的解表示

- 假设有 n 个作业，由于所有作业都必须做完，因此可用一个 n -元组表示问题的解：

$$x = \{x[1], x[2], \dots, x[i], \dots, x[n]\}, x[i] \in \{1, 2, \dots, n\}$$

↑
第 i 步执行的作业

- 对应的解空间是 n 个作业的一个全排列，相当于要从 $n!$ 个排列中求出 n 个作业的一种排列使完成时间最少。

解空间的组织

- 采用回溯法求解，对应的解空间是一个是排列树，从根到叶子的一条路径中相当于一个排列，要求搜索一个排列使完成时间最少的路径。

n 个作业的全排列



完成时间最少

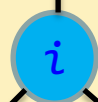
n 个作业的某个排列: 最优解

第1步执行的作业 $x[1]$



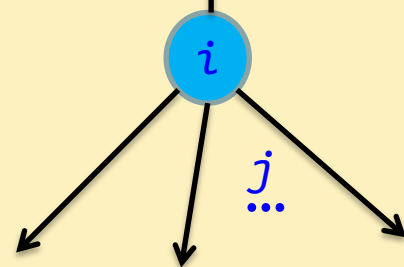
...

第 i 步执行的作业 j : $x[i]=j$



$x[i]$

j
...



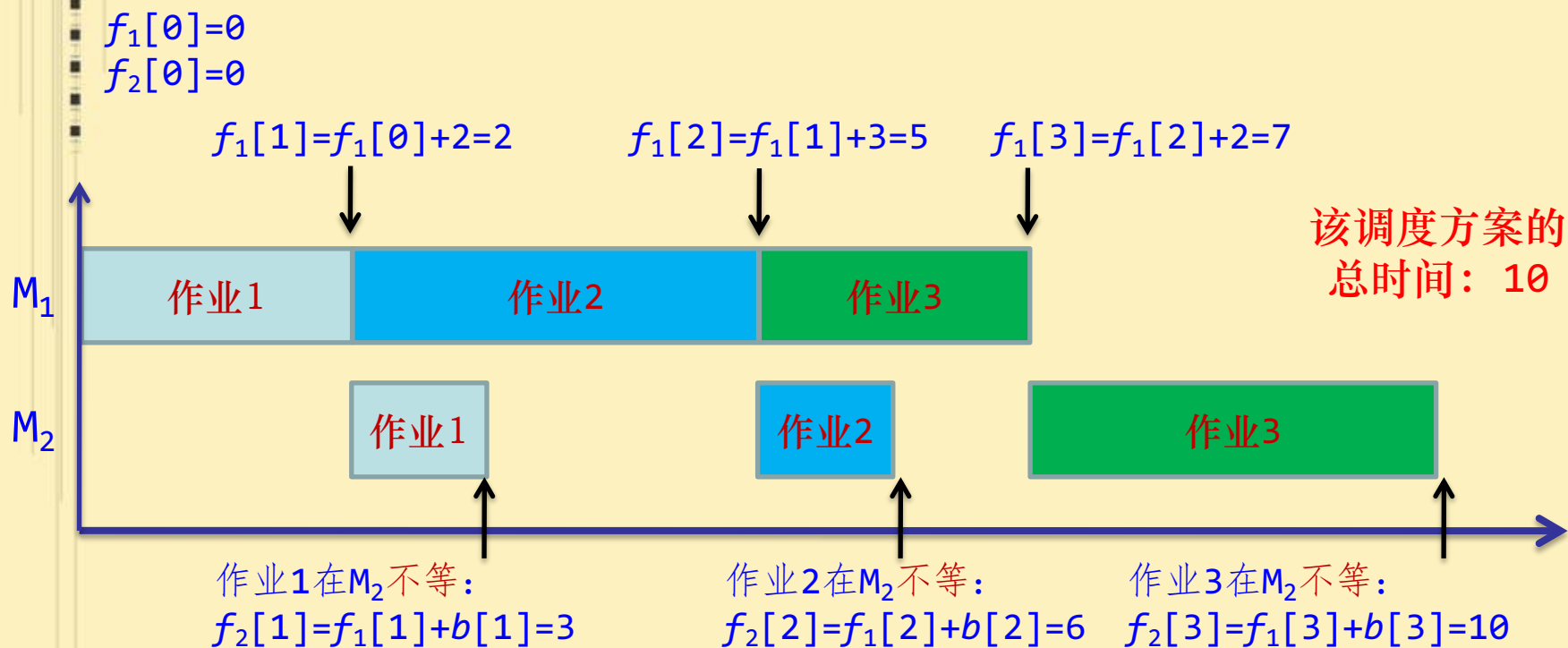
排列的解

- 设引入2个数组，分别记录在M1和M2机器上每一步执行完对应作业后的总时间。
 - f1数组：f1[i]表示第i步执行的作业x[i]在M1上执行完的总时间（含前面作业的执行时间）
 - f2数组：f2[i]表示第i步执行的作业x[i]在M2上执行完的总时间（含前面作业的执行时间）
- 由于一个作业总是先在M1上执行后在M2执行，所以f2[n]就是执行全部作业的总时间。
- 例如：设有3个作业在M1和M2上执行时间为：

作业编号	1	2	3
M ₁ 时间a	2	3	2
M ₂ 时间b	1	1	3

作业编号	1	2	3
M_1 时间 a	2	3	2
M_2 时间 b	1	1	3

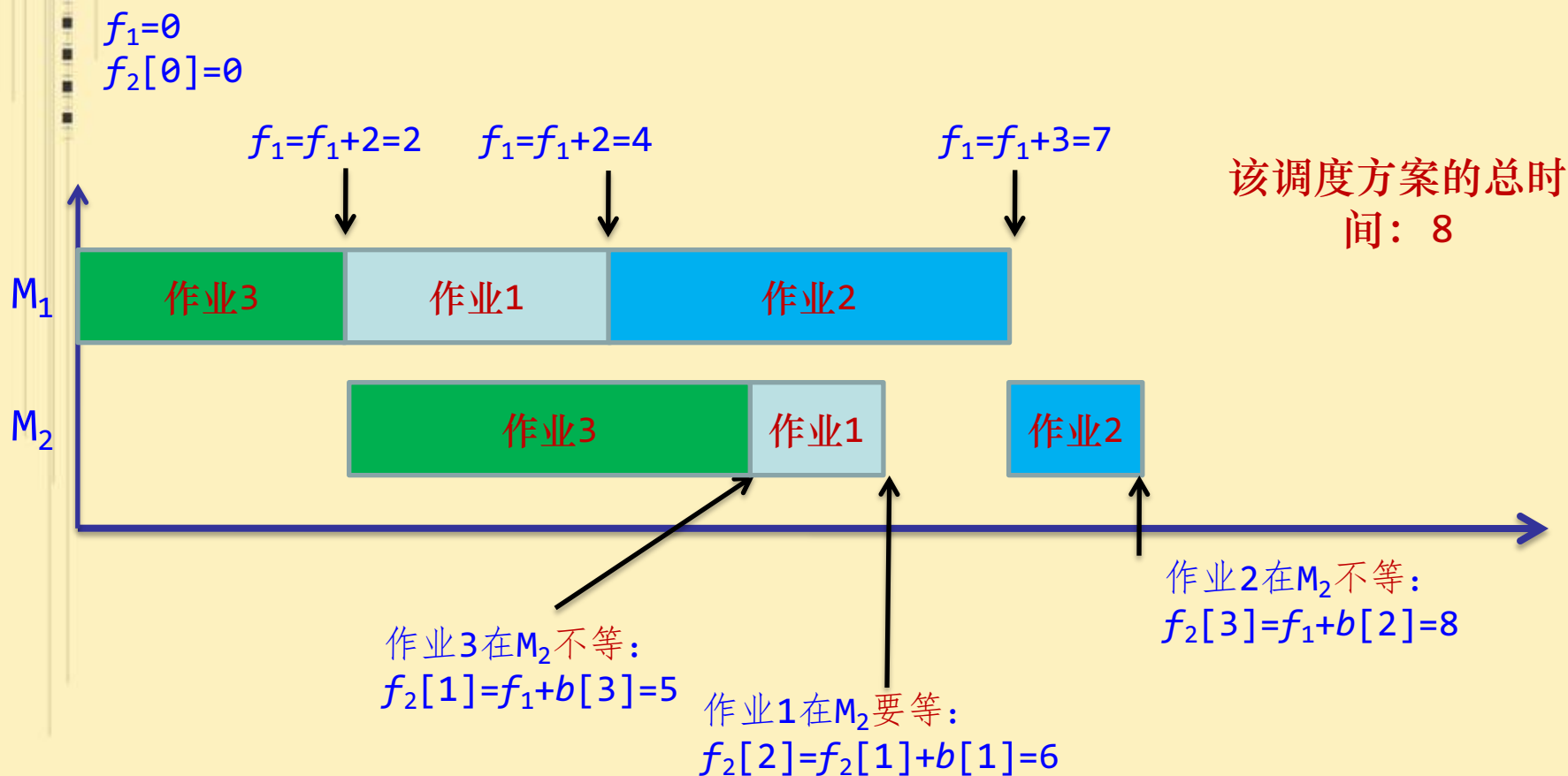
调用方案1: $x=(1, 2, 3)$ ，即按作业1、2、3的顺序执行。

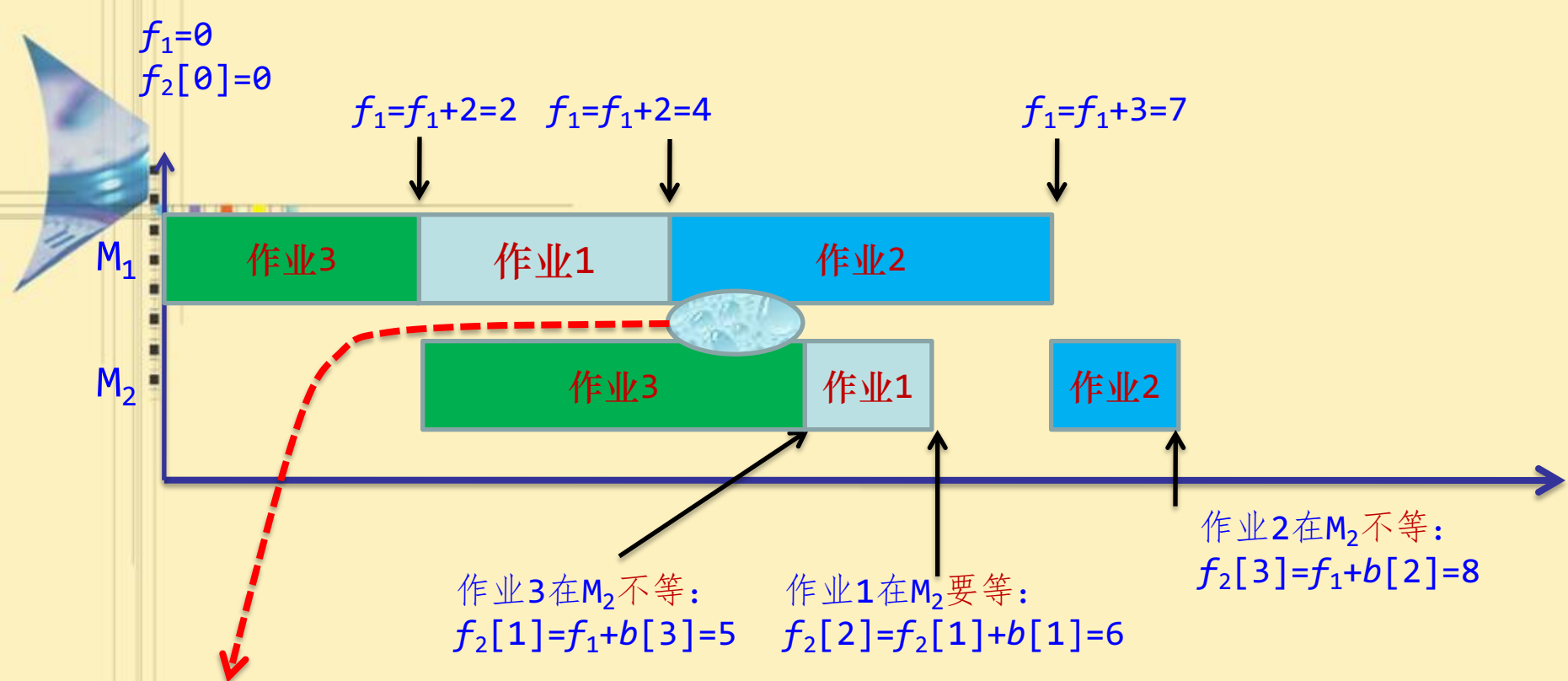


观察：由于每个作业都是从 M_1 开始的，即 M_1 上各个作业是连续执行的，不需要等待，所以 f_1 不需要用数组表示，直接用单个变量 f_1 表示。

作业编号	3	1	2
M_1 时间 a	2	2	3
M_2 时间 b	3	1	1

调度方案2: $x = (3, 1, 2)$ 。





第 i 步执行作业 $x[i]$ 在 M_2 上需要等待的条件: $f_2[i-1] > f_1$; 否则不需要等待

- $f_2[i-1] > f_1$: 需要等待 $\Rightarrow f_2[i] = f_2[i-1] + b[x[i]]$
- $f_2[i-1] \leq f_1$: 不需等待 $\Rightarrow f_2[i] = f_1 + b[x[i]]$

$f_1 += a[x[i]];$

$f_2[i] = \max(f_1, f_2[i-1]) + b[x[i]];$

回溯算法

- 采用排列树递归回溯框架，在求一个方案的同时求其 $f_2[n]$ 时间！
- 剪枝：求出第 i 层的 $f_2[i]$ 即执行作业 $x[i]$ 后的总时间，若 $f_2[i] \geq \text{bestf}$ （ bestf 为当前求出的执行全部作业的最优总时间），就没有必要从该结点向下扩展了，让其成为死结点，也就是说仅仅扩展满足 $f_2[i] < \text{bestf}$ 的结点。

排列树回溯算法基本框架

`x[1..n]` 存放解的向量，并初始化

```
backtrack(int i)
```

```
{  if(i>n)
```

```
    输出结果;
```

```
    else
```

```
    {  for (j=i;j<=n;j++)
```

```
        {  ...
```

```
            swap(x[i],x[j]);
```

```
            if (constraint(i) && bound(i))
```

```
                backtrack(i+1);
```

```
            swap(x[i],x[j]);
```

```
            ...
```

```
        }
```

```
    }
```

```
}
```

//求解排列树的递归框架

//搜索到叶子结点,输出一个可行解

//用j枚举i所有可能的路径

//第i层的结点选择x[j]的操作

//为保证排列中每个元素不同,通过交换来实现

//满足约束条件和限界函数,进入下一层

//恢复状态,回溯

//第i层的结点选择x[j]的恢复操作

算法 JobScheduleREC (递归)

输入: $a[1..n]$, $b[1..n]$: n 个作业在M1和M2上处理时间.

输出: $bestx[1..n]$: n 个作业的最优调度排列, $bestf$: 最优调度总处理时间。

1. **for** $i \leftarrow 1$ **to** n //初始化当前调度

2. $x[k] \leftarrow i$, $f2[i] \leftarrow 0$

3. **end for**

4. $bestf = +\infty$ //初始化 $bestf$

5. $f1 \leftarrow 0$

6. jobschedule (1)

7. **output** $bestx[1..n]$, $bestf$

过程 swap (i, j)

1. $temp \leftarrow x[i]$

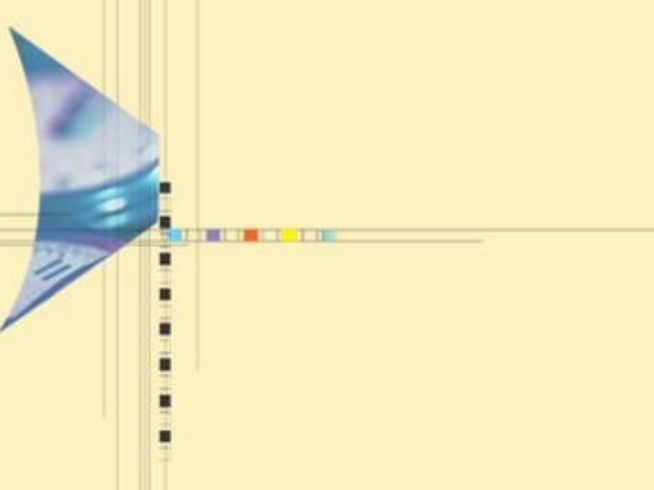
2. $x[i] \leftarrow x[j]$

3. $x[j] \leftarrow temp$


```

过程 jobschedule (i)                                //从第i层开始搜索
1. if (i>n) then                                     //到达叶结点,产生一种调度方案
2.     if (f2[n]<bestf) then                           //找到更优解
3.         bestf  $\leftarrow$  f2[n];
4.         for (j  $\leftarrow$  1 to n)                       //复制解向量
5.             bestx[j]  $\leftarrow$  x[j];
6.         end for
7.     end if
8. else
9.     for(j  $\leftarrow$  i to n)                             //没有到达叶结点,考虑可能的作业
10.        swap(x[i],x[j]);
11.        f1  $\leftarrow$  f1 + a[x[i]]; //选择作业x[i],在M1上执行完的时间
12.        f2[i]  $\leftarrow$  max(f1,f2[i-1])+b[x[i]];
13.        if (f2[i]<bestf) then                         //前进或剪枝
14.            jobschedule(i+1);
15.        end if
16.        f1  $\leftarrow$  f1 - a[x[i]];                         //回溯
17.        swap(x[i],x[j]);
18.    end for
19.end if

```

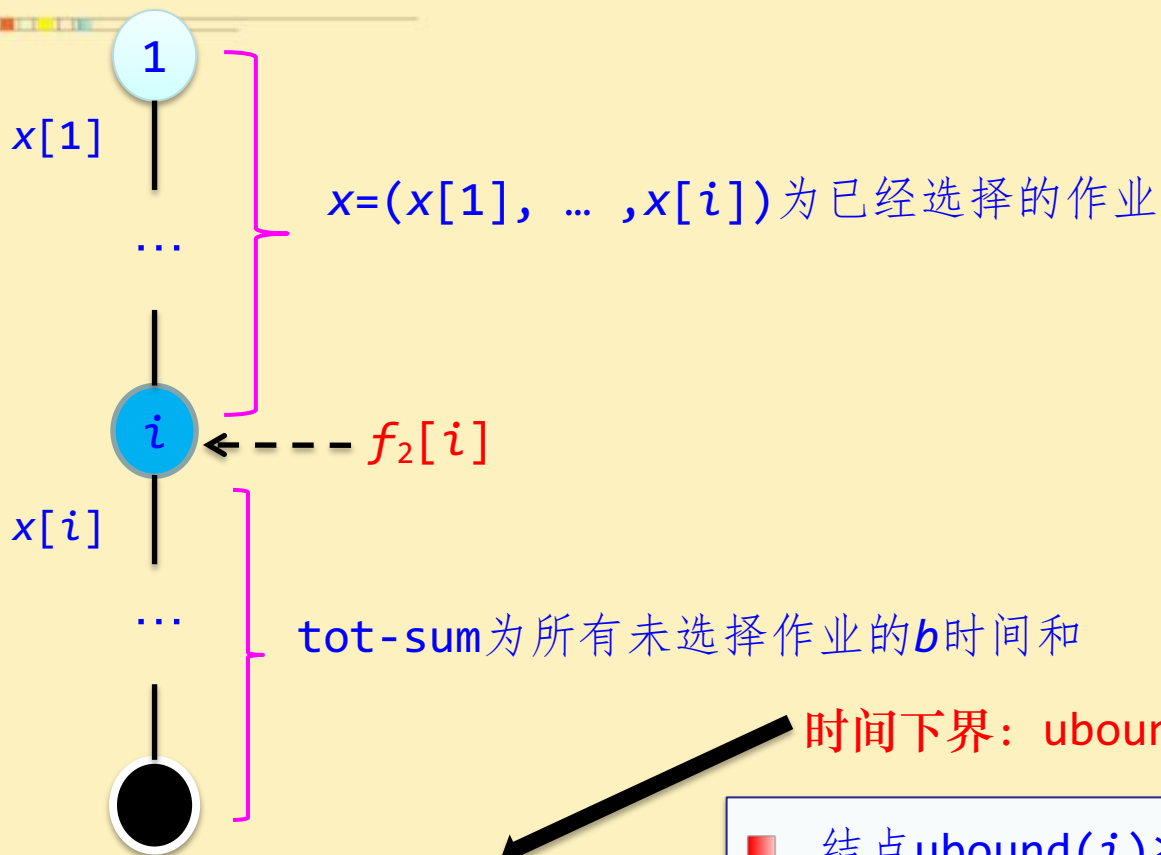



作业编号	1	2	3	4
M1时间 a	5	12	4	8
M2时间 b	6	2	14	7

求解结果：

最少时间：33，最优调度方案：3 1 4 2

进一步剪枝



该调度方案的时间和 $\geq f_2[i] + \text{tot-sum}$

- 结点 $\text{ubound}(i) \geq \text{bestf}$ $\Rightarrow$ 剪枝
- 仅仅扩展 $\text{ubound}(i) < \text{bestf}$ 的结点

结点的下界函数（代价函数）

```
过程 ubound(i)                                //求结点(x[1],...x[i])的下界值
1. tot-sum  $\leftarrow$  0;
2. for (j  $\leftarrow$  i+1 to n)                      //扫描所有未选择的作业
3.   tot-sum  $\leftarrow$  tot-sum+b[x[j]]; //累计所有未选择作业的b时间
4. end for
5. return f2[i]+tot-sum;
```

```

过程 jobschedule (i)                                //从第i层开始搜索
1. if (i>n) then                                     //到达叶结点,产生一种调度方案
2.     if (f2[n]<bestf) then                           //找到更优解
3.         bestf  $\leftarrow$  f2[n];
4.         for (j  $\leftarrow$  1 to n)                       //复制解向量
5.             bestx[j]  $\leftarrow$  x[j];
6.         end for
7.     end if
8. else
9.     for(j  $\leftarrow$  i to n)                             //没有到达叶结点,考虑可能的作业
10.        swap(x[i],x[j]);
11.        f1  $\leftarrow$  f1 + a[x[i]]; //选择作业x[i],在M1上执行完的时间
12.        f2[i]  $\leftarrow$  max(f1,f2[i-1])+b[x[i]];
13.        if (ubound(i)<bestf) then //前进或剪枝
14.            jobschedule(i+1);
15.        end if
16.        f1  $\leftarrow$  f1 - a[x[i]];                       //回溯
17.        swap(x[i],x[j]);
18.    end for
19.end if

```



算法分析

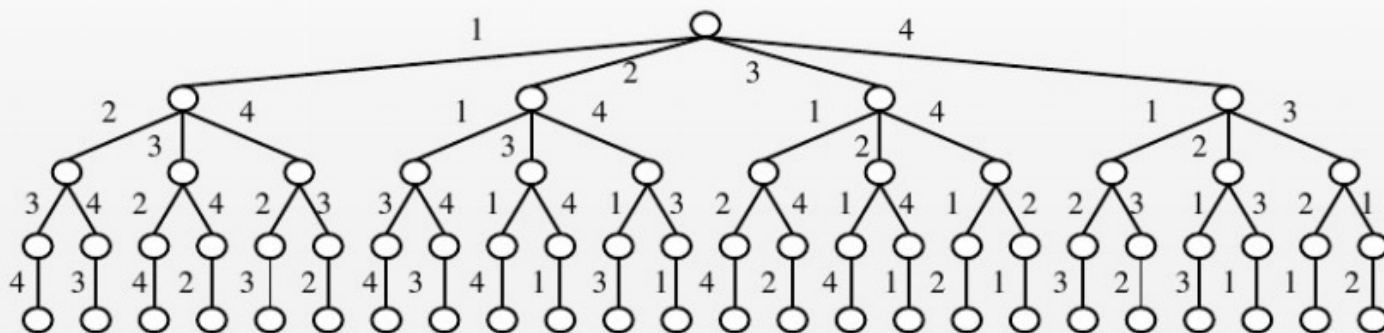
该算法的解空间树是一棵高度为 n 的排列树，算法在每个结点处耗费时间为 $O(1)$ ，因此，在最坏情况下，算法的时间复杂度为 $O(n!)$ 。



8.3.6 货郎担问题

- 设城市个数为 n ，解形式定义为元组 $X=(x_1, x_2, \dots, x_n)$ ，其中每个分量 x_i 的表示第 i 个访问的城市。考虑到每个城市只能访问1次的约束条件，货郎担问题的解空间为所有 n 个城市的全排列，找出其中周游路径最短的排列。

状态空间树：问题解空间的树形式表示
当 $n=4$ 时，货郎担问题的状态空间树。



算法设计

- 设周游路线从 1 开始，则解向量为 $\langle x_1, x_2, \dots, x_{n-1} \rangle$ ，其中 $x_1, x_2, \dots, x_{n-1}$ 为 $\{2, 3, \dots, n\}$ 的排列。搜索空间为 $(n-1)!$ 对应的排列树。
- 结点 $\langle x_1, x_2, \dots, x_k \rangle$ 表示从结点 1 出发得到 k 步的路线
- 约束条件：令 $B = \{x_1, x_2, \dots, x_k\}$ ，则 $x_{k+1} \in \{2, \dots, n\} - B$
- 最优解的下界：当前得到的最短周游路线长度 $bestn$
- 在搜索过程中，利用下界函数来剪枝函数来优化算法搜索。
 - (1) 将当前最优的周游回路长度 $bestn$ ，作为最优解的下界
 - (2) 对已经访问的部分路线 $\langle x_1, x_2, \dots, x_k \rangle$ ，如果部分路线的路径长度超过 $bestn$ ，则剪枝。
 - (3) 如果找到更短的回路，则更新 $bestn$

进一步剪枝

定义结点的代价函数：设顶点 c_i 出发的最短边长度为 l_i ， d_j 为已经选定的周游路线中第 j 段的长度，则

$$L = \sum_{j=1}^k d_j + \sum_{i_j \notin B} l_{i_j}$$

L 为结点对应的已经走过的 k 段周游路线，扩张成全程周游回路的长度的下界值。如果 $L \geq \text{bestn}$ ，则剪枝。

时间 $O(n!)$ ：计算 $O((n-1)!)$ 次，代价函数计算 $O(n)$

预处理步骤计算每个结点出发的最短边，时间为 $O((n-1)!)$